Left shift and Right shift operator

Left Shift(<<)

It is a binary operator that takes two numbers, left shifts the bits of the first operand, and the second operand decides the number of places to shift.

Syntax:

```
x << y;
```

x: First Operand --- data

y: Second Operand ----decides how many positions to shift

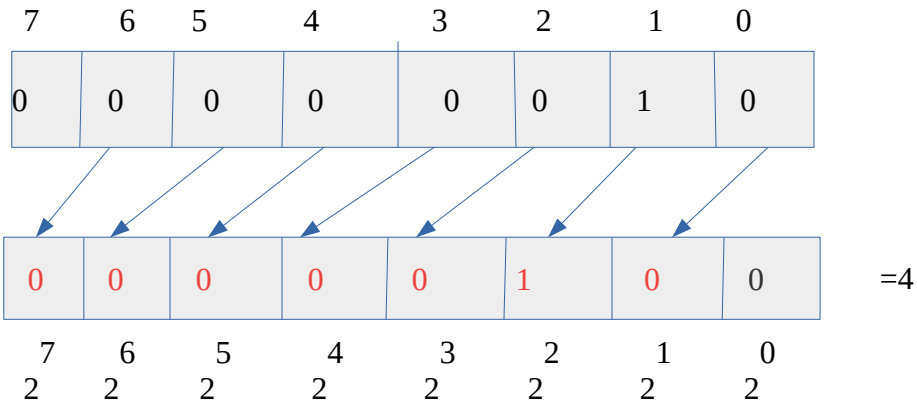
left-shifting an integer '**x**' with an integer **'y'** denoted as '**(x<<y)'** is equivalent to multiplying **a** with **2^y** (2 raised to power y).

int a=2;

a<< 1 = 2 * (2 ^1) =4;

data

number of position to shift



x<< y = x * (2 ^ y)// /// x * (2 power y)

1

2 << 1 = 2 * (2)

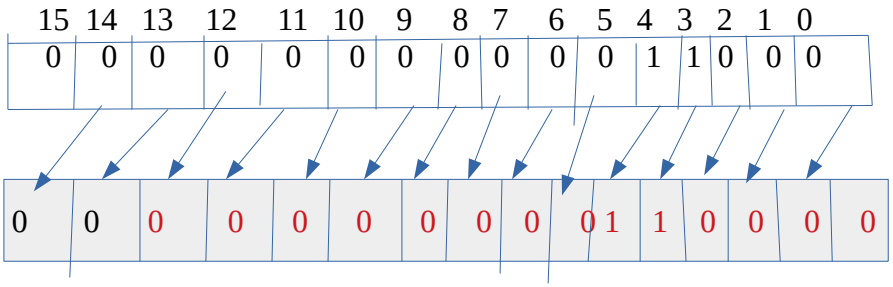
=2 * 2

=4 ////double of x

Q:int a=20,b=0,c=0;
    b=a<<1;
   c=a>>1;

//

short int x=24;
x <<1



result=48

1. 24<<1 = 24 * (2 ^1) =48

2. 24<<2

   24 * (2 ^ 2)= 96
//

task//////verify the output

```
#include<stdio.h>
int main()
{

int a=1 ,b =0, c=-1;

    b= a<<5;
    c=b <<5;
    printf("%d  %d    %d  ",a , b, c);
}
```

//

task: verify the result

```
 int a=1,b=0, c=-1, d=-1, e=-1
 b=a<<10;
c=b<<10;
d= c<<10;
e=d<<1;
```

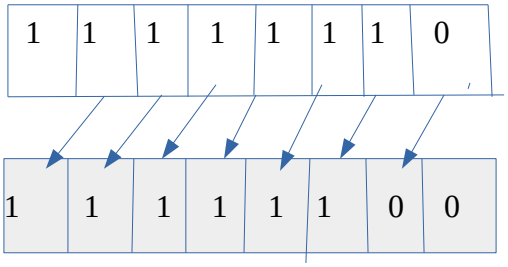
int x=-2

x<<1

Take 2's complement

0 0 0 0 0 0 1 0

```
1   1   1   1   1   1   0   1
+                           1
-----------------------------------------
1   1   1   1   1   1   1   0   = -2
```



?

confirmig the result

```
0   0   0   0   0   0   1   1   +
                                1
-----------------------------------------
0   0   0   0   0   1   0   0       =  -4
```

```
#include<stdio.h>
int main()
{
        int a=-8 ,b=-9 , c=-10;

        printf("%d\n ",a<<1);
        printf("%d\n", b<<1);
        printf("%d\n ", a<<2);
        printf("%d\n",b<<2);


}
```

Right shift operator

Right Shift(>>)

It is a binary operator that takes two numbers, left shifts the bits of the first operand, and the second operand decides the number of places to shift.

Syntax:

```
X >> y;
```

x: First Operand --- data

y: Second Operand ----decides how many positions to shift

left-shifting an integer '**x'** with an integer **'y'** denoted as '**(x<<y)'** is equivalent to divid**ing**

x /(2 raised to power y).

int x=5; which is 101 in Binary Form.

x=x>>1

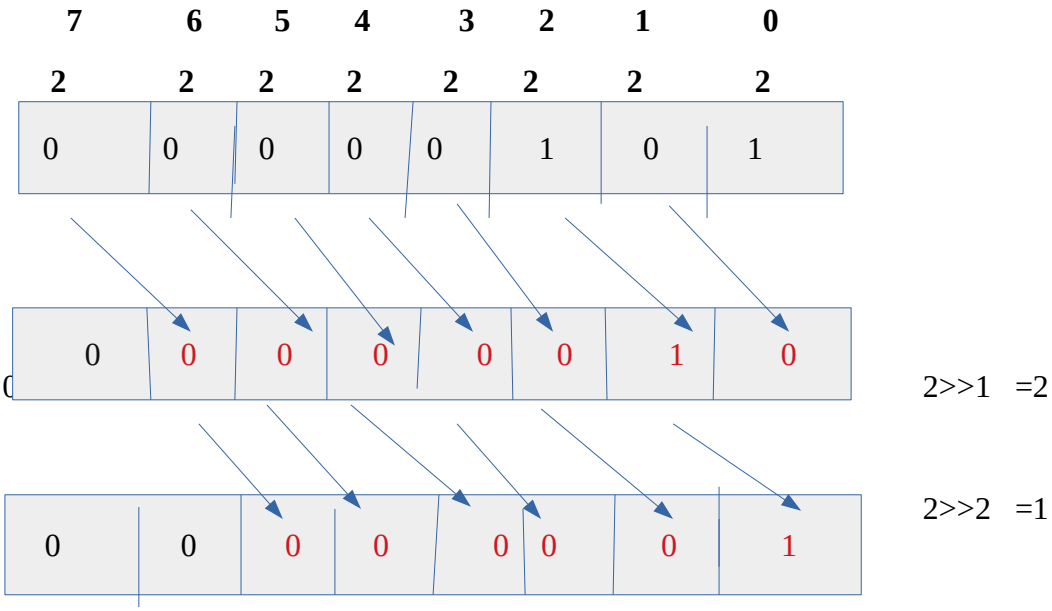
then x will become x=x/(2^1). Thus, x=x/(2^1)=2

x>>2

x= x/ (2^2) , 5/4 =1

//]

q: 2>> 1



Negative value right shifting

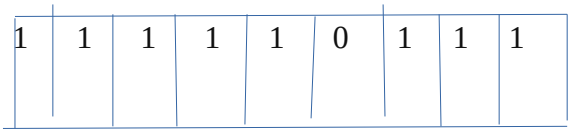
-9 >>1

9= 0 0 0 0 0 1 0 0 1

-9 = 1 1 1 1 1 0 1 1 0

\+ 1

1 1 1 1 1 0 1 1 1 = -9

now -9= 

0 0 0 0 0 1 0 0

\+ 1

---

0 0 0 0 0 1 0 1 -5

//

shortcut

Rights shifting of -ve number=

even number rightshifting----->we get exactly half

odd number ------>half -1

//;

-5 >>4

5 is odd=so take half -1

1. -5>>1

   5/2 = -2 -1

   = -3

2. -3>>1 //0dd(so half -1)

   3/2 = -1 -1

   =-2

3. -2 >>1 ////even==so exactly half

   2/2 =-1

4. -1>>1 ///odd -half -1

   1/2 = -1

result = -1

//

task:

main()

{

int a=-8 ,b =-9;

printf(“% d %d “, a<<1 , a>>1);

printf(“%d   %d “, b<<1 ,b>>1);

}

//

q:int a=20,b=0;

b= a<<3>>2;

printf(“%d   %d “,a ,b);

}

//

int main()

{

  printf(“%d\n “, 1<<5 >>5

 printf("%d\n -15<<1);

 printf(“%d\n”,5>>4);

}

Assignment

1;WAP tos wap two numbers using temporary variable.

2:WAP to swap two numbers without using temporary variable.

3.WAP to Swap Two Numbers Using Bitwise Operator.